

Final Exam

- Due No due date
- Points 100
- Questions 7
- Available May 9 at 12am - May 14 at 11:59pm
- Time Limit 75 Minutes

Instructions

Partial credit may be given for partially correct solutions.

Use correct C++ syntax for writing code.

You are not required to write comments for your code; however, brief comments may help make your intention clear in case your code is incorrect.

This quiz was locked May 14 at 11:59pm.

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	64 minutes	91 out of 100

❗ Correct answers are hidden.

Score for this quiz: 91 out of 100

Submitted May 14 at 11:20am

This attempt took 64 minutes.



The code that follows will be referenced for much of the exam.

```
#include <iostream>
#include <string>
#include <exception>

using namespace std;

class list_exception : public exception {
private:
    const char *msg;
public:
    list_exception(const char *m) : msg(m) {}
    virtual const char* what() const throw()
    {
        return msg;
    }
};

template <class T>
```

```

class linked_list {
protected:
    class list_node {
    public:
        T data;
        list_node *prev, *next;

        list_node(T d, list_node *p, list_node *n) : data(d), prev(p), next(n)
        {
            if (p) p->next = this;
            if (n) n->prev = this;
        }
    } *head, *tail;

public:
    linked_list() : head(0), tail(0) {}

    virtual ~linked_list()
    {
        // To do
    }

    virtual void insert_head(T d)
    {
        head = new list_node(d, 0, head);
        if (!tail) tail = head;
    }

    virtual void insert_tail(T d)
    {
        tail = new list_node(d, tail, 0);
        if (!head) head = tail;
    }

    virtual void insert_sorted(T d)
    {
        // To do
    }

    virtual void remove_head()
    {
        list_node *tmp = head;

        if (!head) {
            throw list_exception("Attempt to remove from empty list.");
        }

        head = head->next;
        delete tmp;
        if (!head)
            tail = 0;
        else
            head->prev = 0;
    }

    virtual void remove_tail()
    {
        list_node *tmp = tail;

        if (!tail) {
            throw list_exception("Attempt to remove from empty list.");
        }

        tail = tail->prev;
        delete tmp;
        if (!tail)
            head = 0;
        else

```

```

        tail->next = 0;
    }

    virtual bool is_empty()
    {
        return !head;
    }

    virtual ostream &print(ostream &o) const
    {
        list_node *cur;

        for (cur = head; cur; cur = cur->next) {
            o << cur->data;
            if (cur->next) o << " -> ";
        }

        return o;
    }
};

template <class T>
ostream &operator<<(ostream &o, const linked_list<T> &l)
{
    return l.print(o);
}

struct dino {
    string name;
    string call;

    dino(string n, string c) : name(n), call(c) {}
};

ostream &operator<<(ostream &o, const dino &d)
{
    return o << d.call;
}

```



Question 1

5 / 5 pts

Give the output of the following program. Please omit newlines from your output.

If there is not output, type *no output*.

If there is an error (run-time or compile-time), do not give any output that may have printed before the error and instead type *error*.

```

int main(int argc, char *argv[])
{
    linked_list<dino> train;

    train.insert_tail(dino("Parasaurolophus", "Herp-derp"));
    train.insert_tail(dino("Archaeopteryx", "Squaaa"));
    train.insert_tail(dino("Yutyranus", "Nom-nom"));
    train.insert_head(dino("Sinosauropteryx", "Meep-mee"));

    cout << train << endl;
}

```

```
    return 0;
}
```

Meep-mee -> Herp-derp -> Squaaa -> Nom-nom



Question 2

5 / 5 pts

Give the output of the following program. Please omit newlines from your output.

If there is not output, type no output.

If there is an error (run-time or compile-time), do not give any output that may have printed before the error and instead type error.

```
int main(int argc, char *argv[])
{
    linked_list<dino> train;

    train.insert_tail(dino("Compsognathus", "Rir-rir"));
    train.insert_tail(dino("Parasaurolophus", "Herp-derp"));
    train.insert_tail(dino("Archaeopteryx", "Squaaa"));
    train.insert_tail(dino("Deinonychus", "Creee"));
    train.insert_tail(dino("Yutyrannus", "Nom-nom"));
    train.insert_tail(dino("Sinosauropteryx", "Meep-mee"));
    train.insert_head(dino("Ankylosaurus", "Mawr"));
    train.insert_head(dino("Apatosaurus", "WHOOOP"));

    cout << train[0] << endl;

    return 0;
}
```

error

Incorrect



Question 3

0 / 5 pts

Give the output of the following program. Please omit newlines from your output.

If there is not output, type no output.

If there is an error (run-time or compile-time), do not give any output that may have printed before the error and instead type error.

```
int main(int argc, char *argv[])
{
    linked_list<dino> train;
```

```

train.insert_tail(dino("Archaeopteryx", "Squaaa"));

try {
    train.remove_tail();
    train.remove_tail();
}
catch (const char *s) {
    cout << "const char *" << endl;
}
catch (const exception &e) {
    cout << "exception" << endl;
}
catch (const list_exception &e) {
    cout << "list_exception" << endl;
}
catch (...) {
    cout << "general" << endl;
}

return 0;
}

```

list_exception



Question 4

20 / 20 pts

Implement the destructor for the **linked_list** class. You are welcome to use any of the methods given in the class or in the STL, but your solution should otherwise be self contained; that is, you may not write any helper methods.

You may assume you are implementing this at the position where it appears in the class body above.

Your Answer:

```

~linked_list(){
    list_node * curr = head;

    while(curr != nullptr){
        list_node * tmp = curr->next;

        delete curr;

        curr = tmp;
    }

    head = nullptr;

    tail = nullptr;
}

```

```
}
```



Question 5

18 / 20 pts

Implement the **insert_sorted** method in the **linked_list** class. You are welcome to use any of the methods given in the class or in the STL, but your solution should otherwise be self contained; that is, you may not write any helper methods.

You may assume you are implementing this at the position where it appears in the class body above.

insert_sorted should insert the parameter **d** into **this** in ascending sorted order. It is the responsibility of the user to ensure that the list was in sorted order before the call; thus, you may assume the list was previously sorted.

Hint: This is a templated class. How do you compare two **T**s? Turns out, it's not really your problem. If you use something that is not implemented in **T**, the compiler will complain to the *user*, not to *you*! That said, you should restrict yourself to operations that you can reasonably expect the developer of **T** to have implemented.

Your Answer:

```
void insert_sorted(T d){  
  
    if (head == nullptr) { // Empty linked list  
  
        head = tail = new list_node(d, nullptr, nullptr);  
  
        return;  
  
    }  
  
    if(d < head->data){  
  
        head = new list_node(d, head, nullptr);  
  
        return;  
  
    }  
  
    if(!(d < tail->data)){  
  
        tail = new list_node(d, nullptr, tail);  
  
        return;  
  
    }  
  
    list_node * curr = head;
```

```

while(curr != nullptr && curr->data < d){

curr = curr->next;

}

new list_node(d, curr, curr->prev);

}

```



Question 6

15 / 15 pts

Given the following class:

```

template <class T>
class linked_stack : public linked_list<T> {
public:
    linked_stack() : linked_list<T>() {}
    void push(T &d)
    {
        // To do
    }
    T pop()
    {
        // To do
    }
};

```

Implement the **pop** method in the **linked_stack** class. You are welcome to use any of the methods given in this class, in **linked_list**, or in the STL, but your solution should otherwise be self contained; that is, you may not write any helper methods.

You may assume you are implementing this at the position where it appears in the class body above.

push (which you do not need to implement!) and **pop** implement standard stack semantics on top of **linked_list**. Your implementation should throw an appropriate exception on any errors.

Note that a stack could be built up from the head or the tail of a doubly-linked list. It doesn't matter in any meaningful way which of these is chosen, and since you are not given an implementation of push to conform to, you may choose either one. In other words, we don't care which end of the list you build your stack on.

Your Answer:

// I am not sure why the push says //TODO in the code block but it says that we dont need to implement push in the question.

// Ill just take this as I dont have to implement push? If i had to implement push its wraps

// Since it says that they dont care which end of the list i build the stack on ill just do head pop.

T pop(){

if(head == nullptr) { // Nothing to pop throws exception (to comply with the requirements in the question)

```
throw list_exception("Attempt to pop from empty list.");
```

}

```
list_node * old = head;
```

T data = old->data;

head = head->next;

if(head != nullptr) {

head->prev = nullptr;

} else {

tail = nullptr;

}

delete old;

return value;

}

// I was hoping there would be the haiku extra credit here. Anyways this is what i wrote for it -

// Malloc gave me hope

// One byte past the promised land

// God left main on read

Partial



Question 7

28 / 30 pts

Evaluate the veracity of the following claims.

1. true C is not a subset of C++.
2. false C++ templates are implemented using type erasure.

3. true The C++ compiler requires template sources at instantiation time.
4. false The only difference between malloc and new is that malloc is a function and new is an operator.
5. true Runtime polymorphism only works on pointers and references.
6. false Since operators are a feature of the language, their use never compiles to a function call.
7. false int cannot be used to instantiate templates.
8. true Inheritance is a static concept.
9. true C and C++ can be linked together.
10. false Destructors are called automatically when pointers go out of scope.
11. true Initialization lists are necessary in order to call non-default constructors on base classes.
12. true Initialization lists are necessary in order to call non-default constructors on instance variables.
13. true C has static functions and static local variables. C++ has both of these as well.
14. true To use a class variable, simply use the static keyword in the class definition.
15. true Objects can be used like functions after you overload the () operator.

Answer 1:

true
false

Answer 2:

true
false

Answer 3:

true
false

Answer 4:

true
false

Answer 5:

true
false

Answer 6:

true
false

Answer 7:

true
false

Answer 8:

true

false

Answer 9:

true

false

Answer 10:

true

false

Answer 11:

true

false

Answer 12:

true

false

Answer 13:

true

false

Answer 14:

true

false

Answer 15:

true

false

Quiz Score: 91 out of 100